

토양지하수처 AX-day | AI 실습 교육

내 업무 하나, 이틀 안에 자동화한다

바이브코딩으로 완성하는 화면 설계 · AI 분석 · DB · 배포 풀 사이클

일정

9 월 3 일 (목) ~ 4 일 (금) 13:00-18:00

장소

산림비전센터

강사

(주) 온토크래프트 이두원 CEO · 양선희 CTO

강사 소개



이두원

(주) 온토크래프트 CEO · 공학박사

- IoT·AI 융합 20 년 + (한국전력공사 전력연구원 , LG 전자)
- 성균관대 · 세종대 겸임교수 , 디자인씽킹 · 혁신 강연 다수
- 산업통상자원부 장관 표창 (2024)
- 저서 : 사물인터넷 비즈니스 모델혁명 외 3 권

담당 양일 오프닝 · 클로징 , Day 2 킥오프 , 발표 심사



양선희

(주) 온토크래프트 CTO

- AI Agent · RAG · Knowledge Graph · 온톨로지 설계
- IT 경력 20 년 + (임베디드 → 엔터프라이즈 AI)
- KRISO 해사 지식그래프 · KOGAS 생성형 AI 플랫폼
- 공정거래위원회 AI 역량강화 교육 (바이트코딩 실습)

담당 전 실습 진행 , 순회 코칭 총괄

이들의 여정 · 완주 그리고 실전

DAY 1

전원 따라하기 · 풀 사이클 완주

- 공통 예제 (수질 데이터 대시보드) 하나로
- PRD → 화면 설계 → AI 분석 → 배포 → DB → 재배포
- 전원이 처음부터 끝까지 직접 완성

DAY 2

조별 과제 · 실전 수행과 발표

- 부서별 실제 업무 과제를 조 단위로 개발
- 역할 분담 협업 + 팀 PRD + 공용 DB
- 배포 URL 시연 발표와 우수 조 선정

교육 종료 시 가져가는 것



배포 URL 로 접속되는
조별 업무 자동화 앱



GitHub 저장소
+ Supabase DB



업무가 문서화된
팀 PRD (진짜 자산)

Day 1 · 오늘 우리가 걷는 길

10:00-12:00	AX 학습동아리 시연회	토양지하수처 자체 세션
13:00-13:40	오프닝 · 과제 구체화	바이브코딩 개요, 라이브 데모, 조별 과제를 “[입력]→[출력]”으로 정의 · 이두원
13:40-14:30	PRD 작성 + 프로젝트 설정	PRD 표준 템플릿, Claude 프로젝트 지시문에 PRD 입력, 샘플 데이터 업로드
14:30-15:30	Claude Design 화면 설계	대화로 대시보드 목업 생성 · 수정 → HTML 내보내기
15:30-16:20	기능 구현 + AI 분석	엑셀 파싱, 기준값 판정, Claude API 해석 연동
16:20-16:50	GitHub · Vercel 1차 배포	웹 업로드 → 배포 → 동료 휴대폰으로 URL 접속 확인
16:50-17:40	Supabase DB 연동	테이블 생성 → 앱 연결 → 새로고침에도 데이터 유지
17:40-18:00	재배포 + 회고	수정 → 자동 재배포 흐름 체득, Day 2 안내

바이브코딩 · 대화로 소프트웨어를 만든다

“자연어로 사람이 **AI 와 대화하며** 소프트웨어를 만드는 것”

기존 방식

- ✗ 개발자에게 의뢰 (외부 용역)
- ✗ 수주 ~ 수개월 소요
- ✗ 비용 수백 ~ 수천만 원
- ✗ 수정 요청 오래 걸림 , 의도 전달 어려움

바이브코딩

- ✓ 담당자가 직접 만든다
- ✓ 수십 분 ~ 수 시간
- ✓ 구독료 · API 비용 수준
- ✓ 즉시 수정 , 내 의도 그대로 반영

왜 지금 바ιβ코딩인가

1 반복 업무가 너무 많다

매주 같은 보고서, 매번 비슷한 집계·검증.
AI 도구 하나로 잃어버린 시간을 되찾습니다.

2 기존 시스템이 업무에 안 맞는다

범용 시스템은 우리 부서 업무의 특수성을 모릅니다.
나만을 위한 100% 맞춤 도구를 직접 만듭니다.

3 아이디어는 있는데 개발자가 없다

‘이런 거 있으면 좋겠다’를 이제 내가 직접,
30 분 만에 프로토타입으로 만듭니다.

4 AI 시대의 새로운 업무 역량

도구를 만드는 사람과 못 만드는 사람의 격차.
지금 시작하면 6 개월 후 업무가 달라집니다.

에이전트의 시대 · 직원과 디지털 인력이 함께 일한다

NVIDIA CEO 젠슨 황

AI 에이전트는 직원과 함께 일하는 ‘디지털 인력’ 이 되며 , 회사에 영입하려면 직원 채용처럼 회사의 언어와 업무 방식에 맞춰 훈련해야 한다고 말합니다 .

엔지니어의 일도 바뀌고 있습니다 . 코드를 짜는 대신 에이전트를 설계하고 , 벤치마크를 만들고 , 가드레일을 세우는 일로 .

에이전트란 무엇인가

- 1 목표를 받으면
- 2 스스로 계획을 세우고
- 3 도구를 사용해
- 4 여러 단계를 실행하고
- 5 결과를 검증한다

그래서 오늘 우리는 어디에 있나

오늘 배우는 것

에이전트를 부리는 법 .
Claude 가 여러분의 첫 디지털 인력

오늘 만드는 것

AI 내장 업무 앱 .
부서 업무의 1 세대 자동화

다음 단계

에이전트 만들기 .
마지막 장의 로드맵에서 소개

이들간 사용하는 도구 · 다섯 개면 충분하다

1 Claude Desktop + 프로젝트

AI 개발 환경

지시문에 PRD 를 넣으면 모든 대화가 설계서 안에서 동작

2 Claude Design

화면 설계

대화로 목업 생성 → 단일 HTML 내보내기

3 Claude API

앱 속 AI 엔진

완성된 앱이 호출하는 해석 · 요약 엔진 (조별 키 제공)

4 Supabase

데이터베이스

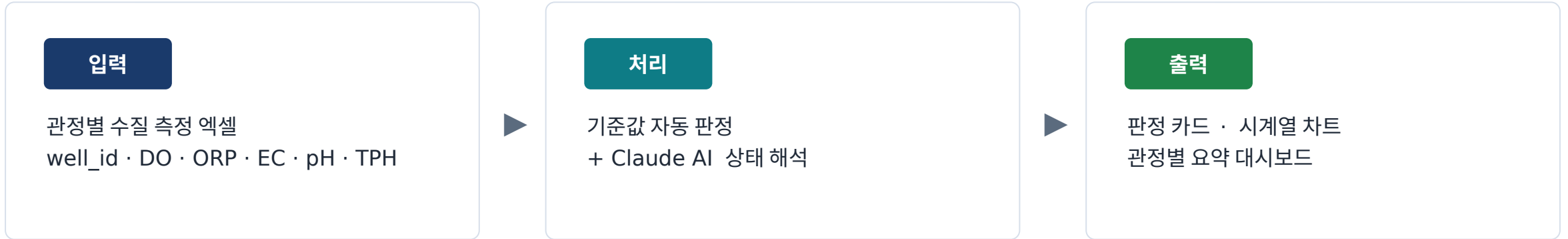
클릭만으로 테이블 생성 , 앱과 연결해 저장 · 조회

5 GitHub + Vercel

저장 · 배포

웹 업로드만으로 저장 , 자동 배포로 URL 생성

공통 예제 · 수질 측정 데이터 AI 분석 대시보드



판정 기준 : 오전 강의의 도메인 지식이 그대로 코드가 된다

TPH > 500 μ g/L

오염 의심

먹는물 기준 초과, 유류 오염
정밀조사

DO < 2 또는 ORP < 0

주의

혐기 · 환원 환경 전환, 오염의
조기 신호

EC $\geq$ 2,000 μ S/cm

주의

이온농도 급증, 용존물질 축적
가능성

DO > 15 mg/L

이상치

유효 범위 초과, 센서 점검 ·
재측정

실습 키트 받기 · 여기서 시작합니다

다운로드 주소

단축 URL 기입 자리 (리허설 후 확정)

QR

휴대폰 카메라로 QR 을 찍어
노트북 메일로 링크를 보내도 됩니다 .

받은 파일은 바탕화면
'AX-day' 폴더에 모아주세요 .

키트 구성 3 가지

1 수질측정 _ 샘플데이터 .xlsx

공통 예제 실습용 더미 데이터 30 건

2 PRD 표준 템플릿

빈칸 채우기 양식 + 수질 예제 작성본

3 교안 PDF

오늘 · 내일 슬라이드 전체 , 복습용

참고 단계별 완성본은 배포하지 않습니다 . 직접 만드는 것이 오늘의 목적이고 , 정말 막힌 조는 프론티어와 강사가 완성본으로 이어드립니다 .

시작 전 4 가지 확인 · 하나라도 안 되면 손 들어주세요

1 Claude Desktop 로그인

앱 실행 → 로그인 확인 (Pro 플랜)
채팅과 Code 탭이 모두 열리는지

2 프로젝트 생성

새 프로젝트 ‘수질분석 - 본인이름’ 생성
지시문 입력창이 보이면 성공

3 Claude Design 열기

사이드바에서 Design 실행
캔버스 화면이 뜨면 성공

4 샘플 데이터 확인

수질측정_샘플데이터.xlsx 다운로드
엑셀에서 열리는지 확인 후 닫기

⚠ 네 가지가 모두 준비되어야 실습 진행이 가능합니다. 막힌 분은 조별 디지털프론티어가 먼저 도와드립니다.

Claude Desktop 설치와 프로젝트 만들기

사전 안내대로 준비하셨다면 이 장은 확인용입니다. 안 된 분은 지금 이 순서대로 따라오세요.

앱 설치와 로그인

- 1 다운로드**
claude.ai/download 접속, 운영체제에 맞는 파일 받기
- 2 설치 실행**
받은 파일 더블클릭 (업무 PC는 관리자 권한 필요할 수 있음)
- 3 로그인**
앱 실행, Google 계정으로 로그인 (Pro 플랜)

프로젝트란 지시문 + 지식 + 대화가 묶인 작업 공간입니다. 대화가 끊겨도, 내일 다시 열어도 같은 규격에서 이어집니다.

프로젝트 만들기

- 1 프로젝트 메뉴**
왼쪽 사이드바에서 '프로젝트' 클릭
- 2 + 새 프로젝트**
화면의 '+ 새 프로젝트' 버튼 클릭 → 이름 입력
(예: 수질분석 - 홍길동) → 만들기
- 3 구조 확인**
지시문 입력창과 지식 (파일 업로드) 영역이 보이면 성공

실제 화면 (리허설 후 스크린샷 교체)
'+ 새 프로젝트' 버튼 위치 표시

계정 3 종 가입 · GitHub 이 가장 먼저인 이유

Vercel 과 Supabase 는 GitHub 계정으로 가입합니다 . 그래서 순서가 GitHub 부터입니다 .

GitHub

- 1 github.com 접속 , Sign up 클릭
- 2 이메일 · 비밀번호 · 아이디 입력 (아이디는 영문 · 숫자만)
- 3 확인 퍼즐 통과 후 인증 메일의 코드 입력
- 4 로그인되면 완료 . 기관 메일은 인증 메일이 늦게 올 수 있습니다

⚠ 가입에 시간이 걸리는 유일한 계정입니다 . 교육 1 주 전 완료를 권한 이유입니다 .

Vercel

vercel.com 접속 → **Continue with GitHub** → Authorize 클릭 .
끝입니다 (약 1 분)

Supabase

supabase.com 접속 → **Sign in with GitHub** → Authorize 클릭 .
역시 1 분입니다

세 서비스 모두 무료 플랜으로 진행합니다 . 결제 정보 입력은 없습니다 .

PRD PRINCIPLE

코드가 아니라 PRD 가 자산입니다

앱이 깨져도 , 담당자가 바뀌어도 잘 쓴 PRD 만 있으면 AI 로 언제든지 다시 만들 수 있습니다 .
오늘 배우는 것은 앱 만들기가 아니라 , 업무를 계속 개발 가능한 상태로 만드는 방법입니다 .

- ✓ 수정이 필요하면 코드가 아니라 PRD 를 고쳐 재생성한다
- ✓ PRD 는 프로젝트 지시문에 넣어 모든 대화의 규격으로 만든다
- ✓ PRD + 샘플데이터 + 최신 파일 , 3 종이면 언제든지 다시 , 더 좋게 만들 수 있다

PRD, 어렵지 않습니다 · 업무 설명 몇 줄이면 초안이 나온다

PRD 생성 요청 (기획실 · 채팅 프로젝트에서)

우리 부서는 관정별 수질 측정 엑셀을 분기마다
취합합니다 . 측정값을 기준과 비교해 이상 여부를
확인하고 상태 요약을 붙여 보고합니다 .
매번 수작업이라 시간이 오래 걸립니다 .

이 업무를 자동화하는 도구의 PRD 를
7 항목 표준 템플릿으로 작성해줘 .

생성된 초안 미리보기 (발췌)

1 목적

“ 수질 측정 엑셀을 넣으면 판정 · 해석
대시보드가 나오는 도구 ”

3 판정 규칙

TPH > 500 → 오염 의심 ...
(기준값은 여러분이 확인 · 수정)

6 완료 기준

“ 샘플 30 건 → 정상 25 · 주의 2 ... ”
(검증 가능한 문장으로)

빈 문서에서 시작하지 않습니다 . 업무를 몇 줄로 설명하고 , 초안을 받고 , 다듬습니다 . 무엇을 기준으로 다듬는지가 다음 장의 7 항목입니다 .

PRD 표준 템플릿 · 7 항목이면 업무가 설계서가 된다

1

목적

“[입력]을 넣으면 [출력]이 나오는 도구” 한 문장

2

입력 / 출력 명세

파일 형식과 컬럼명까지 (영문 , 단위 포함 : TPH_uGL)

3

판정 · 처리 규칙

기준값은 사람이 명문화 , AI 에게 맡기지 않는다

4

AI 역할과 폴백

AI 는 해석 · 요약만 , 키 없으면 규칙 모드 자동 전환

5

기술 제약

단일 HTML, 라이브러리 명시 , 키는 입력창으로

6

완료 기준 (DoD)

“샘플을 넣으면 이 결과” , 검증 가능한 체크리스트

7

유지 · 확장

버전 규칙 + 백로그 + “PRD 를 고쳐 재생성” 원칙

오늘 (Day 1) 은 1~6 번으로 앱을 완성하고 ,
내일 (Day 2) 은 7 번까지 채워 팀의 자산으로 만듭니다 .

역할을 딱 자릅니다 · 숫자 판단은 규칙, 문장 설명은 AI

규칙이 합니다 · 사람이 정하고 코드가 실행

판정

TPH 가 500 을 넘었는가 ?
→ 넘으면 ‘오염 의심’ , 색상 뱃지까지 규칙이 결정

기준 관리

기준을 바꾸고 싶다 ?
→ 코드가 아니라 PRD 3 번의 기준값을 수정

안전장치

AI 가 응답하지 못하면 ?
→ 미리 정해둔 규칙 문구로 자동 전환 (폴백)

AI(Claude) 가 합니다 · 문장으로 설명

해석

이 관정의 상태를 어떻게 설명할까 ?
→ 판정 결과를 받아 행정 문서 어조로 해석문 작성

요약 · 제안

석 달치 추세가 어떤가 ?
→ 시계열을 읽고 요약과 후속 조치를 제안

한계

AI 가 판정을 바꿀 수 있나 ?
→ 없습니다 . 해석만 하고 판정에는 손대지 못함

구분법은 한 줄입니다 . 답이 숫자 비교로 나오면 규칙 , 문장으로 나오면 AI. 판단을 AI 에 맡기지 않는 것이 환각을 막는 설계입니다 .

완료 기준 (DoD) · “ 다 됐나요 ? ” 를 AI 에게 묻지 마라

수정을 몇 번을 반복해도 , 이 체크리스트만 통과하면 앱은 깨지지 않은 것입니다 .

오늘 예제의 완료 기준은 이미 정해져 있습니다 . 여러분은 넣고 , 확인만 하면 됩니다 .

- ☐ 수질측정 _ 샘플데이터 .xlsx 를 넣으면 30 건 = 정상 25 · 주의 2 · 이상치 1 · 오염 의심 2
- ☐ W01 의 DO 25.0 행이 ‘이상치’ 로 표시된다 (유효 범위 초과)
- ☐ W02 의 TPH 320 행이 ‘주의’ 로 표시된다 (기준 미만이지만 DO 1.8, 조기 신호)
- ☐ Supabase 저장 후 새로고침해도 데이터가 유지된다
- ☐ 배포 URL 에서 위 항목이 모두 동일하게 동작한다

핵심 여러분 조의 과제에도 내일 이런 완료 기준을 직접 씁니다 . 검증 기준이 있는 업무만 계속 개발할 수 있습니다 .

PRD 5 대 원칙 · 이것만 지키면 계속 개발할 수 있다

1 PRD 가 자산이다

코드는 언제든지 다시 만들 수 있습니다 . 남겨야 할 것은 요구사항 정의서입니다 .

2 업무 설명에서 시작한다

빈 문서가 아니라 , 업무를 몇 줄로 설명하고 초안을 받아 다듬습니다 .

3 판단은 규칙 , 설명은 AI

기준은 사람이 명문화하고 코드가 실행합니다 . AI 는 해석만 말합니다 .

4 완료는 기준으로 검증한다

“ 다 됐나요 ” 를 AI 에게 묻지 않습니다 . 검증 가능한 체크리스트로 확인합니다 .

5 수정은 PRD 를 고쳐 재생성한다

문제의 주소는 늘 PRD 안에 있습니다 . 디자인은 1 번 , 판정은 3 번 , 저장은 2 번 .

다섯 원칙의 공통점은 하나입니다 . 사람이 기준을 쥐고 , AI 에게 일을 시킨다 .

프로젝트 지시문에 PRD 넣기 · 하네스를 거는 순간



이렇게 하면 무엇이 달라지나

- ✓ 이후 모든 대화가 PRD 규격 안에서 동작 · 매번 요구사항을 다시 설명할 필요가 없다
- ✓ “PRD 대로 만들어줘” 한 줄로 재생성 가능 · 대화가 끊겨도, 내일 다시 열어도 이어진다
- ✓ 담당자가 바뀌면 프로젝트째 넘기면 끝 · 인수인계의 자동화

기획실과 작업실 · 오늘 우리는 두 개의 방을 쓴다

기획실

채팅 프로젝트

- PRD 를 쓰고 다듬는 곳
- 지시문에 PRD 를 걸어두고
요구사항을 대화로 정리
- 방금 여러분이 있던 방



작업실

Claude Code

- 파일을 만들고 고치는 곳
- 폴더 안에서 index.html 을
생성 · 수정 · 실행
- 지금부터 실습이 이뤄지는 방

확정한 PRD 를 PRD.md 파일로 저장해 작업 폴더에 넣는 순간 , 기획이 작업실로 넘어갑니다 . 기획과 구현을 방으로 나누는 이 습관 자체가 하네스입니다 .

작업실 열기 · Code 탭과 작업 폴더

작업실 여는 3 단계

1 Code 탭 클릭

Claude Desktop 왼쪽에서 Code 선택

2 작업 폴더 만들기 · 열기

바탕화면에 my-project 폴더 생성 후
Code 에서 이 폴더를 열기

3 화면 확인

왼쪽에 파일 목록, 가운데 대화 입력창.
대화로 파일을 만들고 고치는 구조

작업 폴더 표준 구조

```
my-project/  
PRD.md           ← 기획실에서确定的 설계서  
design.html       ← Design 에서 내보낸 화면  
수질측정_샘플데이터.xlsx  
index.html       ← Code 가 만들 완성 앱
```

팁 PRD 를 CLAUDE.md 이름으로 저장하면 Code 가 매번 자동으로 읽습니다.
하네스 자동 장착.

⚠️ 폴더 이름은 my-project 처럼 영문으로, 한글 · 공백 지양. 파일은 탐색기에서 폴더로 끌어다 넣으면 됩니다.

실행 화면 (리허설 후 스크린샷 교체)

Code 탭 실제 화면

전체 지도 · 모든 단계의 입력은 PRD 다



그래서 **PRD** 가 제일 중요합니다. 디자인이 이상하면 1 번을, 판정이 틀리면 3 번을, 저장이 안 되면 2 번을 고치면 됩니다. 문제의 주소가 늘 PRD 안에 있습니다.

Claude Design · 말이 곧 스케치가 되는 캔버스

무엇인가

대화로 화면을 그리는 디자인 도구입니다. 코드도, 디자인 툴 사용법도 필요 없습니다. 원하는 화면을 말로 설명하면 시안이 그려지고, “카드를 더 크게” 한마디면 그 자리에서 바뀝니다.

여는 법

Claude Desktop 사이드바에서 **Design** 클릭
또는 브라우저에서
claude.ai/design 접속 (Pro 기능)

화면 구조 · 왼쪽에서 말하면 오른쪽에 그려진다

채팅

요청 입력
수정 지시



캔버스

시안이 실시간으로 그려짐
클릭해서 요소 지정 가능

1 요청

만들고 싶은 화면을
문장으로 설명



2 확인

캔버스에 그려진
시안 확인



3 수정

대화로 고치기,
한 번에 한 가지



4 내보내기

HTML 로 저장,
프로젝트 지식에 업로드

⚠ 베타 기능이라 화면 배치가 이 자료와 조금 다를 수 있습니다. 다르게 보이면 손 들어주세요, 함께 확인합니다.

말로 그리는 대시보드 · 코드 없이 화면부터

DESIGN 프롬프트

토양지하수처에서 쓸 ‘수질 측정 데이터 AI 분석 대시보드’ 화면을 설계해줘 .

- 상단 : 판정 카드 4 개 (전체 / 정상 / 주의 / 오염 의심)
- 중단 : 관정별 TPH 시계열 차트 + 판정 분포 도넛
- 하단 : 측정 데이터 표 (판정 배지 , AI 해석 컬럼)
- 공공기관 남색 테마 (#1A3A6B), 배경 #EEF2F7
- 엑셀 업로드 영역 포함

진행 방법

- 1 캔버스에 목업이 그려지면 대화로 수정 (“ 카드를 더 크게” , “ 차트를 나란히”)
- 2 마음에 들면 HTML 내보내기
- 3 내보낸 파일을 my-project 폴더에 design.html 이름으로 저장

확인 포인트 화면은 ‘시안’ 입니다 . 완벽하게 다듬는 데 시간을 쓰지 말고 , 구조가 잡히면 바로 다음 단계로 . 디자인 미세조정은 기능이 붙은 뒤에도 언제든지 가능합니다 .

캔버스에 이 구조가 보이면 성공

실행 화면 (리허설 후 스크린샷 교체)

Claude Design 캔버스에 그려진 대시보드 목업

확인 포인트

- ✓ 상단에 판정 카드 4 개 영역
- ✓ 중단에 차트 2 개 영역
- ✓ 하단에 데이터 표
- ✓ 엑셀 업로드 영역

세부 모양은 사람마다 달라도 됩니다 .
네 영역이 보이면 다음 단계로 .

다음 두 가지까지 완료 ① HTML 내보내기 파일이 다운로드됨 ② design.html 이름으로 my-project 폴더에 넣음 . 안 되는 분은 손 들어주세요 .

design.html 을 Code 에 넘기고 , 이해했는지 먼저 묻는다

전달 3 단계

1 파일 옮기기

다운로드 폴더의 내보내기 파일을
my-project 로 끌어와 design.html 로 이름 변경

2 Code 에서 확인

왼쪽 파일 목록에 design.html 이
보이는지 확인

3 기획서 동거 확인

PRD.md 가 같은 폴더에 있는지 확인 .
둘이 같이 있어야 구현이 정확해집니다

이해 확인 프롬프트 (Claude Code 에서)

폴더의 PRD.md 와 design.html 을 읽고 ,
무엇을 만들게 될지 3 줄로 요약해줘 .

→ 요약이 기획 의도와 맞으면 구현 시작

→ 어긋나면 PRD 를 고치고 다시 요약

왜 묻고 시작하나 신입에게 일을 시킬 때와 같습니다 . 이해를 확인하고 시작하면
재작업이 사라집니다 .

디자인에 심장을 달다 · 엑셀 파싱 · 판정 · AI 해석

구현 프롬프트 (Claude Code · 작업실에서)

폴더의 design.html 과 PRD.md 를 읽고,
동작하는 앱을 index.html 로 만들어줘.

- SheetJS 로 엑셀 업로드 → 표에 표시
- PRD 3 번 판정 규칙대로 정상 / 주의 / 이상치 / 오염 의심
 배지를 자동 표시
- Chart.js 로 TPH 시계열 (기준선 500) + 판정 도넛
- Claude API 로 판정별 상태를 60 자 요약
 (키는 입력창으로, 없으면 규칙 문구로 풀백)
- 같은 폴더에 저장해줘

실행 화면 (리허설 후 스크린샷 교체)

판정 배지 · 차트가 표시된 대시보드

폴더의 index.html 더블클릭 실행

Code 가 만든 파일을 브라우저로 열어
샘플 데이터를 직접 넣어봅니다.

원칙 한 번 요청에 한 가지만 바꾼다 · 여러 개를 한꺼번에 고치면 다른 곳이 망가집니다.

막히면 : 뒤쪽 ‘에러 6 종’ 장 참조, 조 프론티어 호출

STEP 2

결과 확인 · 완료 기준 대조

숫자가 맞으면 성공 · 30 건, 넷으로 갈린다

25

정상

2

주의

1

이상치

2

오염 의심

W02 관정의 이야기 · 오전에 배운 오염 패턴이 그대로 보인다

DO 급감

4.5 → 1.8

산소 부족

ORP 음전환

+88 → -65

환원 환경

EC 급증

352 → 780

용존물질 축적

TPH 검출

0 → 1,200

유류 오염

TPH 320 인 달은 기준 (500) 미만인데도 ‘주의’ · DO 가 먼저 신호를 줬기 때문입니다 . 기준을 넘기 전에 잡아내는 것 , 이것이 규칙 설계의 힘입니다 .

대화로 앱을 키우는 법 · 한 번에 한 가지

디자인 수정

- “판정 카드를 더 크고 눈에 띄게”
- “모바일에서도 잘 보이게 반응형으로”

기능 추가

- “엑셀 다운로드 버튼 추가해줘”
- “관정별 필터 드롭다운 추가해줘”

AI 동작 개선

- “요약을 40 자 이내로 줄여줘”
- “후속 조치 제안을 한 줄 덧붙여줘”

안정화

- ▶ “API 오류 시 규칙 모드로 자동 전환”
- “파일 형식이 다르면 안내 메시지”

동작 확인된 버전은 이름을 바꿔 저장하고 다음 수정으로 . app_v1.html → v2 → v3 · 언제든지 되돌아갈 수 있는 안전판입니다 .

명령어 없이 , 웹 화면만으로 저장한다

1 저장소 생성

github.com 로그인 → 우상단 + →
New repository → 이름 입력 (영문)
→ Public 선택 → Create

2 파일 올리기

저장소 페이지 → Add file →
Upload files → 완성한 HTML 을
드래그 & 드롭

3 업로드 확정

하단 초록색
Commit changes 클릭
→ 저장 완료

⚠ **보안 절대 원칙** API 키가 코드 안에 하드코딩되어 있으면 GitHub 에 올리지 마세요 . 우리 앱은 키를 입력창으로 받는 구조라 안전합니다 . 이 구조를 지키는 것이 PRD 5 번 (기술 제약) 의 이유입니다 .

GitHub 계정은 사전 안내로 전원 가입 완료 상태입니다 . 로그인 안 되는 분은 지금 손 들어주세요 .

파일과 폴더, 정확히 이렇게 올린다

파일 몇 개만 올릴 때

- 1 저장소 페이지 → Add file → Upload files
- 2 파일들을 드래그 & 드롭 (여러 개 동시 가능)
- 3 하단 초록색 Commit changes 클릭

폴더째 올릴 때 (my-project 전체)

- 1 같은 Upload files 화면에서
- 2 폴더 자체를 통째로 드래그 → 하위 파일과 구조가 그대로 올라갑니다
- 3 Commit changes 클릭

규칙 세 가지

- ✓ index.html 이 저장소 최상위에 있어야 배포 시 첫 화면이 됩니다
- ✓ 파일 · 폴더 이름은 영문 권장, 키나 비밀번호가 든 파일은 올리지 않습니다
- ✓ 수정본 재업로드도 같은 방법입니다. 같은 이름이면 덮어쓰기 → Vercel 자동 재배포

내 앱에 주소가 생긴다 · 폰으로 접속해보세요

1

Vercel 로그인

vercel.com →
Continue with GitHub

2

프로젝트 가져오기

Add New → Project →
방금 만든 저장소 Import

3

배포

설정 변경 없이
Deploy 클릭 (1~2 분)

4

URL 생성 완료

내앱 .vercel.app,
누구나 접속 가능

지금, 옆 사람과 URL 을 교환하세요

휴대폰 브라우저로 서로의 대시보드에 접속해봅니다. 방금까지 노트북 안에만 있던 도구가 지금은 주소를 가진 서비스입니다. 외주 없이, 서버 없이, 오늘 오후에 만든 것입니다.

이 순간이 바뀌는 지점입니다. ‘만들어 달라는 사람’ 에서 ‘만드는 사람’ 으로.

GitHub 이 안 되면 · Code 에게 배포를 맡긴다

언제 쓰나

GitHub 가입, 로그인, 업로드
어디에서든 막혔을 때.

이 길로 가도 도착지는 같습니다.
접속 가능한 내 앱의 URL.

절차 3 단계

1

Code 에게 말하기

아래 프롬프트 한 줄이면 시작됩니다

2

처음 한 번 로그인

화면에 뜨는 안내를 따라 브라우저에서
Vercel 로그인 (이메일 가입도 가능,
GitHub 계정이 없어도 됩니다)

3

URL 확인

배포 완료 주소가 표시되면
휴대폰으로 접속 확인

배포 프롬프트

이 폴더를 Vercel 로
배포해줘.

(수정한 뒤에는)
다시 배포해줘.

GitHub 경로와의 차이 자동 재배포가 없어서 수정할 때마다 “다시 배포해줘” 한마디가 필요합니다. 그 외 결과물은 동일합니다.

어느 길이든 도착지는 같습니다. 오늘, 전원이 배포에 성공합니다.

Supabase · 데이터가 사는 집

무엇인가

클라우드 데이터베이스 서비스입니다. 지금까지 만든 앱은 브라우저를 닫으면 데이터가 사라집니다. DB 에 저장하면 언제 열어도, 어느 컴퓨터에서 열어도, 팀원이 열어도 같은 데이터가 있습니다.

앱과 연결하는 열쇠 2 개

Project URL (우리 DB 의 주소) + **anon key** (출입증)
이 2 개를 복사해두면 앱이 DB 를 찾아갑니다

대시보드에서 쓰는 곳은 딱 두 군데

Table Editor

엑셀처럼 생긴 화면.
테이블 만들기, 쌓인
데이터 눈으로 확인

Settings · API

Project URL 과
anon key 가 있는 곳.
복사해서 앱에 입력

1 프로젝트 생성

리전 Seoul 로
새 프로젝트

2 테이블 만들기

Table Editor 에서
클릭만으로

3 열쇠 복사

URL 과 anon key 를
메모장에

4 앱과 연결

앱 입력창에 붙여넣기,
저장 · 조회 확인

내일 미리보기 조별 협업에서는 팀 Supabase 1 개를 만들어 열쇠를 공유합니다. 모든 팀원의 화면이 같은 DB 를 바라보는 것, 그게 협업입니다.

내 데이터베이스를 만든다, 하나씩

- 1 로그인 supabase.com → Sign in with GitHub
- 2 New Project 초록색 New Project 버튼 클릭
- 3 이름 입력 Name: water-db (영문 · 숫자)
- 4 DB 비밀번호 Generate a password 클릭 → 생성된 값을 메모장에 보관 (앱 연결엔 안 쓰지만 관리에 필요)
- 5 리전 선택 Region: Northeast Asia (Seoul)
- 6 생성 Create new project → 준비에 1~2 분 , 기다립니다

실행 화면 (리허설 후 스크린샷 교체)

New Project 입력 화면

⚠ 준비 중 (Setting up) 일 때 다음 단계로 넘어가면 오류가 납니다 .
초록불이 켜질 때까지 대기 .

클릭만으로 만드는 **water_quality** 테이블

1

Table Editor

왼쪽 메뉴에서 표 모양 아이콘 클릭

2

New table

이름 : water_quality (정확히 이 철자)

3

RLS 해제

Enable Row Level Security 체크 해제
(교육용 , 실무 적용 시엔 켜고 설정)

4

컬럼 추가

Add column 을 눌러 오른쪽 표의 9 개를
하나씩 추가 (이름 · 타입 정확히)

5

저장

Save → 목록에 테이블이 보이면 성공

추가할 컬럼 9 개

well_id text

measure_date date

ec_uscm · do_mgl · orp_mv · ph · tph_ugl

→ 다섯 개 모두 numeric

judgment · ai_summary text

id, created_at 은 자동 생성 열입니다 . 건드리지 말고 그대로 두세요 .

실행 화면 (리허설 후 스크린샷 교체)

Table Editor 컬럼 추가 화면

검수 옆 사람과 컬럼명을 서로 대조하세요 . 오타 하나가 저장 실패의 90% 입니다 .

주소와 출입증, 두 개만 챙기면 된다

1 설정 열기

왼쪽 맨 아래 톱니 (Project Settings) → API

2 주소 복사

Project URL (<https://xxxx.supabase.co>)
→ 메모장에 붙여넣기

3 출입증 복사

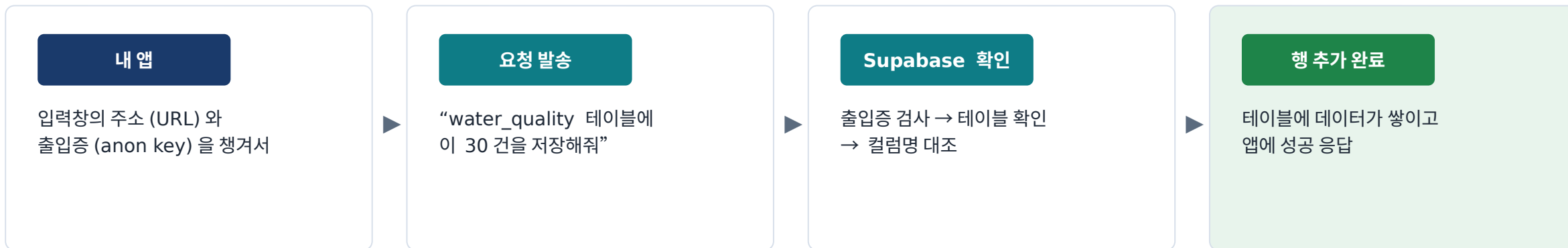
Project API keys 에서 anon public 키
→ 메모장에 붙여넣기

실행 화면 (리허설 후 스크린샷 교체)

Settings → API 화면 (URL·anon key 위치)

⚠ **service_role** 키는 절대 복사·사용·공유 금지. 모든 보안을 우회하는 관리자용 만능키입니다. 우리가 쓰는 것은 anon public 하나뿐입니다.

저장 버튼을 누르면 무슨 일이 일어나나



불러오기는 같은 길의 반대 방향입니다. 주소와 출입증만 맞으면 어느 컴퓨터의 앱이든 같은 테이블을 봅니다. 이것이 내일 조별 협업의 원리입니다.

왜 원리를 알아야 하나 · 저장 실패의 90% 가 이 세 가지

- ✓ 주소 오타 → 요청이 도착하지 못함 (URL 재확인)
- ✓ 출입증 누락 · 오복사 → 문전박대 (anon key 재확인)
- ✓ 컬럼명 불일치 → 짐을 못 내림 (테이블과 PRD 2 번 대조)

저장 버튼 하나 , 그리고 새로고침의 순간

연동 프롬프트 (Claude Code 에서)

분석 결과를 Supabase 에 저장하고 , 앱을 열 때 저장된 데이터를 불러와 표시하는 기능을 추가해줘 .

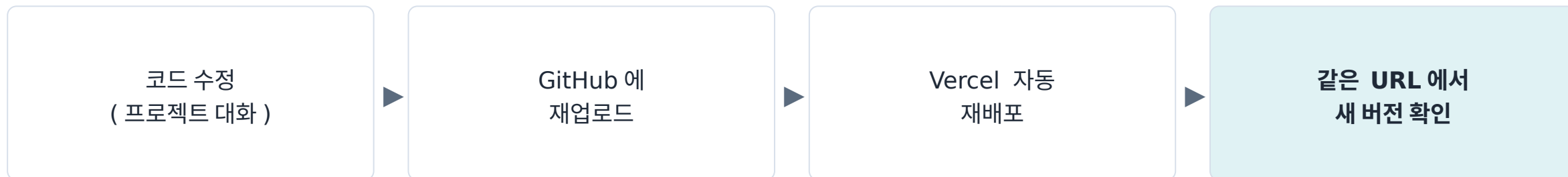
- 테이블 : water_quality (컬럼은 PRD 참고)
- Project URL 과 anon key 는 입력창으로 받기
- ‘DB 에 저장’ / ‘DB 에서 불러오기’ 버튼
- 저장 성공 시 건수 안내 메시지

확인 의식 · 세 번의 검증

- 1 앱에서 ‘DB 에 저장’ → 성공 메시지
- 2 브라우저 새로고침 → 데이터가 남아 있다
- 3 Supabase 대시보드에서 실제 쌓인 행을 눈으로 확인

“ 내 앱이 진짜 데이터베이스에 쓰고 있다 .” 엑셀 파일을 주고받던 업무와 , 같은 DB 를 여러 사람이 바라보는 업무 . 내일 조별 협업에서 이 차이가 결정적입니다 .

재배포 · 수정하면 저절로 배포되는 구조



DB 연동 버전을 GitHub 에 다시 올리면 Vercel 이 알아서 재배포합니다 . 앞으로 개선은 이 한 바퀴의 반복입니다 . 오늘 이후에도 , 여러분 자리에서도 .

Day 1 회고 · 오늘 완주한 것

- ✓ PRD 를 쓰고 프로젝트 지시문에 걸었다 (하네스)
- ✓ 화면 설계 (Design) → 기능 + AI 분석 → 배포 URL → DB 저장 → 재배포 , 풀 사이클 완주
- ✓ 내일 : 같은 사이클을 우리 조의 진짜 업무로 . 샘플 데이터와 과제 한 줄 , 준비되어 있죠 ?

DAY 2

이제 , 여러분의 업무 차례입니다

어제는 같은 예제를 다 같이 완주했습니다 . 오늘은 조별로 진짜 과제를 만듭니다 .
역할을 나누고 , 팀 PRD 를 세우고 , 발표에서 배포 URL 로 시연합니다 .

13:00

조별 킥오프

13:40

구현 스프린트

16:20

통합 + 배포

16:50

조별 발표

17:40

마무리

Day 2 · 오늘 우리가 걷는 길

10:00-12:00

AX 학습동아리 시연회

토양지하수처 자체 세션 (12:50 재집합)

13:00-13:40

조별 킥오프

과제 확정, 역할 분담, 팀 PRD 작성, 전원 지시문 적용 · 이두원

13:40-16:20

구현 스프린트

역할별 병렬 작업, 조 공용 Supabase 공유 (15:20 중간 체크인 : 동작하는 1 차 버전)

16:20-16:50

통합 + 배포

통합 담당 파일 취합, GitHub 업로드, Vercel 배포로 조 URL 생성

16:50-17:40

조별 발표

조당 10~12 분 : URL 라이브 시연 + 절감 효과 + 활용 계획, 거수 투표

17:40-18:00

마무리

우수 조 선정, 실무 적용 보안 안내, 후속 과정 소개

역할 분담 · 여섯 명이 한 팀으로 움직이는 법

PRD 오너 · 1 명

선임 권장

요구사항 정의, 팀 PRD 관리,
수정 요청의 최종 결정

도구 프로젝트 지시문

데이터 담당 · 1 명

샘플 데이터 · 컬럼 정의,
Supabase 테이블 설계 · 구축

도구 Supabase

화면 담당 · 2 명

각자 화면 1 개씩 설계 · 구현
(예 : 입력 화면 / 대시보드)

도구 Claude Design + 프로젝트

통합 · 배포 담당 · 1 명

팀원 파일 취합, GitHub 업로드,
Vercel 배포, 발표 시연

도구 GitHub · Vercel

+ 디지털프론티어 1 명 : 조 안의 기술 조력자 : 막히면 프론티어 먼저, 그다음 강사 호출

충돌 없는 협업 · 세 가지 원칙

1

같은 PRD, 각자 프로젝트

PRD 오너가确定的한 팀 PRD 를 전원이 각자 프로젝트 지시문에 붙여넣습니다 .
같은 설계서 , 각자의 작업 공간 .
누가 시켜도 비슷한 결과가 나오는지 직접 확인하게 됩니다 .

2

한 파일 = 한 사람

화면 담당 A 는 입력 화면 , B 는 대시보드 .
자기 담당 파일만 수정합니다 .
서로의 작업을 덮어쓸 일이 원천적으로 없습니다 .

3

팀 Supabase 는 하나

데이터 담당이 프로젝트 1 개를 만들고 접속 정보를 조 대화방에 공유 .
모든 화면이 같은 DB 를 바라봅니다 .
협업이란 같은 데이터를 공유하는 것입니다 .

branch 도 merge 도 없습니다 . 구조로 충돌을 막는 것이 초보 팀의 가장 빠른 협업입니다 .

네 개의 조, 네 개의 진짜 과제

1 조 · 토양지하수 계획부

경영평가 실적 관리 자동화

실적 엑셀 → 자동 집계 + 대시보드

2 조 · 토양조사부

이력정보시스템 DB 자동검증

CSV → 누락 · 형식오류 · 이상치 검증 리포트

3 조 · 토양정화부

과제 확정 (킥오프에서 최종)

사전 제출된 과제 정의 기준으로 진행

4 조 · 지하수관리부

안심지하수 자동화 에이전트

수질 해석 · 지점 선정 · 작업일보 초안

네 과제 모두 어제 예제와 같은 뼈대입니다 : 정형 데이터 입력 → 판정 · 집계 → 화면 · 문서 출력 . 여러분은 이미 한 번 만들어봤습니다 .

킵오프 · 40 분 안에 네 가지를 확정한다

1 과제 확정 · 5 분

“[입력]을 넣으면 [출력]이 나오는 도구” 한 문장으로 . 욕심내지 않기 , 오늘 안에 시연 가능한 범위로

2 역할 분담 · 5 분

분담표에 이름 기입 : PRD 오너 / 데이터 / 화면 A·B / 통합 · 배포 (+ 프론티어)

3 팀 PRD 작성 · 25 분

템플릿 7 항목 전부 · 특히 6 번 완료 기준 (발표 때 검증할 체크리스트) 과 7 번 유지 · 확장 (교육 후 계획)

4 전원 지시문 적용 · 5 분

완성된 팀 PRD 를 각자 프로젝트 지시문에 붙여넣기 → 스프린트 시작

킵오프 종료 = 팀 PRD 가 전원의 프로젝트에 들어간 상태 . 여기까지 되면 스프린트는 병렬로 굴러갑니다 .

구현 스프린트 · 막힘 없이 달리는 법

15:20 중간 체크인

전 조 ‘동작하는 1 차 버전’ 확인 .
완벽이 아니라 동작이 기준입니다 .

막히면 3 단계 호출

- ① 조 프론티어 → ② 순회 강사 →
- ③ 그래도 안 되면 단계별 완성본으로 이어가기

한 요청 한 가지

어제의 원칙 그대로 .
동작 버전은 v1, v2 로 저장하며 전진

에러는 메시지째 복사

빨간 에러 메시지를 통째로 붙여넣고
“ 이 에러 고쳐줘 ” . 에러가 곧 해답입니다

스프린트의 적은 어려운 기능이 아니라 ‘완벽하게 하려는 마음’ 입니다 . 동작 먼저 , 다듬기는 그다음 .

자주 만나는 에러 6 종 · 메시지가 곧 해답

화면이 하얗게 나옴

F12 → Console 탭의 빨간 메시지를 통째로 복사 → “이 에러 고쳐줘”

401 Unauthorized

API 키 오류 : sk-ant 로 시작하는지 , 복사 중 공백이 섞였는지 확인

429 Too Many Requests

요청 과다 : 1~2 분 대기 후 재시도 . 조별 키를 나눠 쓰는 이유입니다

엑셀 업로드 안 됨

엑셀 프로그램에서 파일이 열려 있으면 닫기 , 확장자 · 암호 확인

Supabase 저장 실패

URL·anon key 재확인 , 테이블 컬럼명이 PRD 와 일치하는지 확인

에러는 아닌데 결과가 이상함

PRD 를 더 구체적으로 고쳐 재생성 , “[이 부분] 만 바꿔줘” 로 좁히기

보안 세 가지 약속 · 오늘도, 실무에서도

1

키는 코드에 넣지 않는다

Claude API 키 · Supabase 키 모두
입력창으로 .
교육용 API 키는 조별 지급, 종료 후 즉시
폐기됩니다 .

2

교육 데이터는 더미만

실데이터는 반입하지 않습니다 .
컬럼 구조만 같고 값은 가상인 데이터로 .
구조가 같으면 도구는 그대로 작동합니다 .

3

실무 적용 전 보안 설정

Supabase RLS · 인증, 기관 보안 검토를
거친 뒤 적용 .
오늘 만든 앱은 ‘검증된 프로토타입’ .
실전 투입 전 마지막 관문이 있습니다 .

이 세 가지가 **PRD 5 번 (기술 제약)**에 항상 들어가야 하는 이유입니다 . 보안은 습관이 아니라 설계입니다 .

통합과 배포 · 조의 결과물이 하나의 주소가 된다

1 파일 취합

화면 담당들이 완성본을
조 대화방에 전달
→ 통합 담당이 수합

2 GitHub 업로드

조 저장소에 파일 일괄 업로드
(index.html = 첫 화면)
→ Commit changes

3 Vercel 배포

Import → Deploy
→ 조 URL 생성
→ 전 조원 폰에서 최종 점검

발표 전 최종 점검 · 팀 PRD 6 번 (완료 기준) 으로 셀프 검증

- ✓ 배포 URL 에서 샘플 데이터 업로드 → 기대 결과가 나오는가
- ✓ 저장 → 새로고침 → 데이터 유지되는가
- ✓ 발표 시연 시나리오 1 회 리허설 (누가 무엇을 보여줄지)

발표 · 세 가지 질문에 답하면 된다

1

어떤 불편함이었나

이 업무에 매주 몇 시간이 들어갔는지,
무엇이 가장 번거로웠는지

2

무엇을 만들었나

배포 URL 접속 라이브 시연.
실제 흐름대로 입력 → 판정 → 결과

3

앞으로 어떻게 쓸 것인가

예상 절감 시간 (주당 몇 시간),
팀 PRD 7 번의 확장 계획

심사와 선정

- ✓ 심사 : 이두원 CEO 강평 + 동료 거수 투표 (가장 ‘내일부터 쓰고 싶은’ 앱에 한 표)
- ✓ 기준 : 완성도보다 실무 절감 효과와 지속 가능성 (PRD 7 번)
- ✓ 우수 조 : 후속 코칭 우선 지원

오늘 가져가는 진짜 성과물은 앱이 아닙니다

여러분의 업무가 문서화된 PRD, 그것이 자산입니다

앱은 다시 만들 수 있습니다 . PRD + 샘플데이터 + 최신 파일 3 종이 있는 한 , 담당자가 바뀌어도 업무는 계속 개발 가능한 상태로 남습니다 . 부서에 이런 PRD 가 쌓이는 것 , 그것이 AX 입니다 .

교육이 끝난 뒤에도

계속 개선하기

PRD 수정 → 재생성 → GitHub 업로드 → 자동 재배포 . 오늘 배운 한 바퀴의 반복

실무 적용 절차

보안 설정 (RLS·인증) + 기관 검토 후 투입 .
필요 시 후속 코칭으로 지원

다음 단계 학습

Python 분석 자동화 , IoT 연계 등
심화 과정 , 별도 안내 예정

내 API 키 만들기 · 교육이 끝나도 앱이 살아있으려면

오늘 쓴 교육용 키는 종료 후 폐기됩니다. 다음 주에도 앱의 AI 기능을 쓰려면 내 키가 필요합니다.

발급 4 단계 (약 5 분)

- 1 콘솔 접속**
console.anthropic.com, Google 계정으로 로그인
- 2 키 생성**
왼쪽 메뉴 API Keys → 우상단 Create Key 클릭
→ 키 이름 입력 (예 : 부서명 - 업무명) → 생성
- 3 키 보관**
sk-ant 로 시작하는 키를 복사해 안전한 곳에.
창을 닫으면 다시 볼 수 없습니다
- 4 결제 · 크레딧 확인**
해외결제 가능 카드 등록 · 무료 크레딧 제공
여부와 금액은 콘솔에서 직접 확인

콘솔 화면 (리허설 후 스크린샷 교체)
API Keys 메뉴와 Create Key 버튼 위치 표시

알아둘 것 세 가지

- ✓ **쓴 만큼 과금** · 콘솔에서 사용량 확인 습관
- ✓ **키는 비밀번호** · 코드 · GitHub 금지, 입력창 방식 유지
- ✓ **부서 공용 키** · 관리 책임자 1 명 지정, 재발급 · 폐기 권한 일원화

발급이 어려우면 후속 코칭에서 부서 단위 발급 · 관리 체계를 함께 잡아드립니다. 혼자 막혀서 앱을 놓지 마세요.

오늘의 앱 , 내일의 에이전트 · 성장 로드맵

1 세대 · 오늘 완성

AI 내장 앱

사람이 파일을 올리고 버튼을 누르면
AI 가 판정하고 해석하는 도구 .
여러분이 오늘 배포한 바로 그 앱

2 세대 · 다음 단계

자동 실행

정해진 시간에 스스로 데이터를 읽고
분석 · 보고서 작성 , 이상 발견 시 알림 .
사람은 결과만 확인

3 세대 · 목표

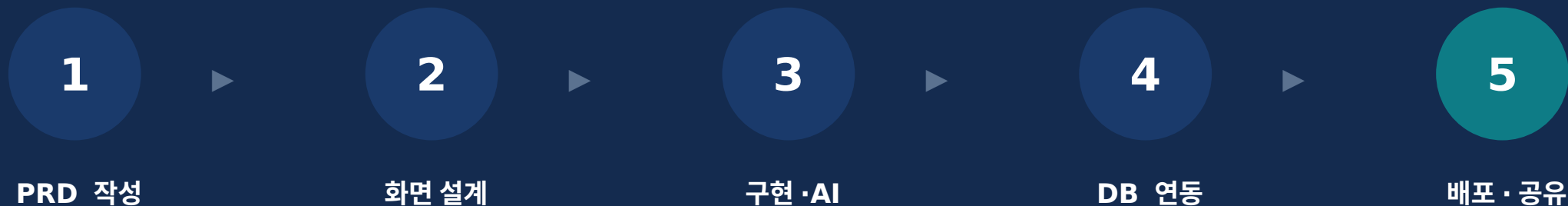
업무 에이전트

목표를 주면 계획을 세우고 도구를 써서
여러 단계를 실행 , 부서 시스템과 연계 .
젠슨 황이 말한 ‘디지털 인력’

여러분의 PRD 가 그대로 에이전트의 하네스가 됩니다 . 지시문 , 판정 규칙 , 완료 기준 , 폴백 . 에이전트에게 필요한 설계 문서를 여러분은 이미 쓸 줄 압니다 .

Thanks

이틀간 수고하셨습니다 . 이제 여러분은 만드는 사람입니다 .



Q & A | (주) 온토크래프트 이두원 CEO · 양선희 CTO